DR. B.R. AMBEDKAR NATIONAL INSTITUTE OF TECHNOLOGY, JALANDHAR

AI – 526

Concurrent Programming Laboratory

Practical File

Submitted To:

Dr. Ruchika Arora

Submitted By:

Atul Dhiman

25901325

A.I. (M.Tech)

```
# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

from numba import cuda
import numpy as np

@cuda.jit
def hello_kernel():
    # Thread and block indices
    tx = cuda.threadIdx.x
    bx = cuda.blockIdx.x
    bdim = cuda.blockDim.x

    # Global thread ID
    gid = bx * bdim + tx
    print("Hello from Block", bx, "Thread", tx, "Global ID", gid)

blocks = 2
threads_per_block = 4
hello_kernel[blocks, threads_per_block]()
cuda.synchronize()
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:680: NumbaPerformanceWarning: Grid size 2 will likely
result in GPU under-utilization due to low occupancy.
  warn(NumbaPerformanceWarning(msg))

Hello from Block 1 Thread 0 Global ID 4
Hello from Block 1 Thread 1 Global ID 5
```

```
Hello from Block 1 Thread 2 Global ID 6
Hello from Block 1 Thread 3 Global ID 7
Hello from Block 0 Thread 0 Global ID 0
Hello from Block 0 Thread 1 Global ID 1
Hello from Block 0 Thread 2 Global ID 2
Hello from Block 0 Thread 3 Global ID 3
```

```
# This Python 3 environment comes with many helpful analytics
libraries installed
# It is defined by the kaggle/python Docker image:
https://github.com/kaggle/docker-python
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/"
directory
# For example, running this (by clicking run or pressing Shift+Enter)
will list all files under the input directory

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/)
that gets preserved as output when you create a version using "Save &
Run All"
# You can also write temporary files to /kaggle/temp/, but they won't
be saved outside of the current session

from numba import cuda
import numpy as np

@cuda.jit
def add_2d_arrays(A, B, C):

    row = cuda.blockIdx.y * cuda.blockDim.y + cuda.threadIdx.y
    col = cuda.blockIdx.x * cuda.blockDim.x + cuda.threadIdx.x

    # Boundary check
    if row < A.shape[0] and col < A.shape[1]:
        C[row, col] = A[row, col] + B[row, col]

rows, cols = 4, 4

A = np.arange(rows * cols, dtype=np.float32).reshape(rows, cols)
B = np.ones((rows, cols), dtype=np.float32)
C = np.zeros((rows, cols), dtype=np.float32)

d_A = cuda.to_device(A)
d_B = cuda.to_device(B)
d_C = cuda.to_device(C)

threads_per_block = (16, 16)   # (x, y)
blocks_per_grid_x = (cols + threads_per_block[0] - 1) //
threads_per_block[0]
```

```
blocks_per_grid_y = (rows + threads_per_block[1] - 1) //
threads_per_block[1]

blocks_per_grid = (blocks_per_grid_x, blocks_per_grid_y)

add_2d_arrays[blocks_per_grid, threads_per_block](d_A, d_B, d_C)
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:680: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(NumbaPerformanceWarning(msg))
```

```
C = d_C.copy_to_host()

print("Matrix A:\n", A)
print("Matrix B:\n", B)
print("Result C:\n", C)
```

```
Matrix A:
 [[ 0.  1.  2.  3.]
 [ 4.  5.  6.  7.]
 [ 8.  9. 10. 11.]
 [12. 13. 14. 15.]]
Matrix B:
 [[1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]
 [1. 1. 1. 1.]]
Result C:
 [[ 1.  2.  3.  4.]
 [ 5.  6.  7.  8.]
 [ 9. 10. 11. 12.]
 [13. 14. 15. 16.]]
```

```
from numba import cuda
import numpy as np

@cuda.jit
def matmul_2d(A, B, C):
    # Global row and column
    row = cuda.blockIdx.y * cuda.blockDim.y + cuda.threadIdx.y
    col = cuda.blockIdx.x * cuda.blockDim.x + cuda.threadIdx.x

    # Check bounds
    if row < C.shape[0] and col < C.shape[1]:
        temp = 0.0
        for k in range(A.shape[1]):
            temp += A[row, k] * B[k, col]
        C[row, col] = temp

M, K, N = 4, 3, 5

A = np.random.rand(M, K).astype(np.float32)
B = np.random.rand(K, N).astype(np.float32)
C = np.zeros((M, N), dtype=np.float32)

d_A = cuda.to_device(A)
d_B = cuda.to_device(B)
d_C = cuda.to_device(C)

threads_per_block = (16, 16)

blocks_per_grid_x = (N + threads_per_block[0] - 1) //
threads_per_block[0]
blocks_per_grid_y = (M + threads_per_block[1] - 1) //
threads_per_block[1]

blocks_per_grid = (blocks_per_grid_x, blocks_per_grid_y)

matmul_2d[blocks_per_grid, threads_per_block](d_A, d_B, d_C)
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:697: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(NumbaPerformanceWarning(msg))
```

```
C = d_C.copy_to_host()

print("Matrix A:\n", A)
print("Matrix B:\n", B)
print("Matrix C = A x B:\n", C)
```

```
Matrix A:
 [[0.5259653  0.8956204  0.7957479 ]
 [0.65585846 0.2610976  0.35645932]
```

```
 [0.45664984 0.43301323 0.7733074 ]
 [0.32504573 0.12558906 0.57478356]]
Matrix B:
 [[0.43875012 0.1574042  0.13247311 0.7035236  0.50905704]
 [0.17113882 0.780277   0.7296751  0.57215524 0.46694425]
 [0.00237093 0.1578071  0.6782411  0.26513997 0.58350265]]
Matrix C = A x B:
 [[0.3859294  0.90719587 1.2628971  1.0934474  1.1502721 ]
 [0.33328703 0.36321515 0.5191654  0.7053119  0.66378236]
 [0.276294   0.53178227 0.9009416  0.7740494  0.88588077]
 [0.1654698  0.23986274 0.52454084 0.45293188 0.55949765]]
```

```
print("NumPy Result:\n", A @ B)
```

```
NumPy Result:
 [[0.3859294  0.9071958  1.262897   1.0934474  1.1502721 ]
 [0.33328703 0.36321515 0.5191654  0.7053119  0.66378236]
 [0.27629402 0.53178227 0.9009416  0.7740494  0.88588077]
 [0.1654698  0.23986275 0.52454084 0.45293188 0.55949765]]
```

Convert RGB to greyscale

```
from numba import cuda
import numpy as np
import time

start = time.time()
@cuda.jit
def rgb_to_grayscale(rgb_img, gray_img):
    row = cuda.blockIdx.y * cuda.blockDim.y + cuda.threadIdx.y
    col = cuda.blockIdx.x * cuda.blockDim.x + cuda.threadIdx.x

    if row < rgb_img.shape[0] and col < rgb_img.shape[1]:
        r = rgb_img[row, col, 0]
        g = rgb_img[row, col, 1]
        b = rgb_img[row, col, 2]

        gray_img[row, col] = 0.299 * r + 0.587 * g + 0.114 * b

img_path = "/content/Endgane_image.jpg"

import cv2
import matplotlib.pyplot as plt
image = cv2.imread(img_path)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
plt.imshow(image_rgb)
plt.axis("off")
```

```
(np.float64(-0.5), np.float64(539.5), np.float64(249.5), np.float64(-
0.5))
```


```
# Output grayscale image
gray = np.zeros((image.shape[0], image.shape[1]), dtype=np.float32)

d_image = cuda.to_device(image)
d_gray = cuda.to_device(gray)
threads_per_block = (16, 16)

blocks_per_grid_x = (image.shape[1] + threads_per_block[0] - 1) //
threads_per_block[0]
blocks_per_grid_y = (image.shape[0] + threads_per_block[1] - 1) //
threads_per_block[1]

blocks_per_grid = (blocks_per_grid_x, blocks_per_grid_y)
rgb_to_grayscale[blocks_per_grid, threads_per_block](d_image, d_gray)
gray = d_gray.copy_to_host()
print(gray.shape)
```

```
(250, 540)
```

```
gray = d_gray.copy_to_host()
print(gray.shape)
```

```
(250, 540)
```

```
import matplotlib.pyplot as plt


plt.imshow(gray, cmap='gray')
plt.title("Grayscale Image")
plt.axis('off')

plt.show()
```


```
end = time.time()
print("Execution time:", end - start, "seconds")
```

```
!pip install numba opencv-python matplotlib
```

```
Requirement already satisfied: numba in
/usr/local/lib/python3.12/dist-packages (0.60.0)
Requirement already satisfied: opencv-python in
/usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: matplotlib in
/usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in
/usr/local/lib/python3.12/dist-packages (from numba) (0.43.0)
Requirement already satisfied: numpy<2.1,>=1.22 in
/usr/local/lib/python3.12/dist-packages (from numba) (2.0.2)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7-
>matplotlib) (1.17.0)
```

```
import numpy as np
import cv2
import os
import math
from numba import cuda
import matplotlib.pyplot as plt

print("CUDA Available:", cuda.is_available())
```

```
CUDA Available: True
```

```
IMG_SIZE = 28

def load_data(path):
    images = []
    labels = []
```

```
    for label, folder in enumerate(["mask", "unmask"]):
        folder_path = os.path.join(path, folder)

        for file in os.listdir(folder_path):
            img_path = os.path.join(folder_path, file)
            img = cv2.imread(img_path, cv2.IMREAD_GRAYSCALE)

            if img is None:
                continue

            img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
            img = img.astype(np.float32) / 255.0

            images.append(img)
            labels.append(label)

    return np.array(images), np.array(labels)


X, y = load_data("dataset")
print("Total Images:", len(X))
print("Shape:", X.shape)
```

```
Total Images: 40
Shape: (40, 28, 28)
```

```
# ---------------- Convolution ----------------
@cuda.jit
def conv2d(input_img, kernel, output):
    x, y = cuda.grid(2)

    if x < IMG_SIZE-2 and y < IMG_SIZE-2:
        s = 0.0
        for i in range(3):
            for j in range(3):
                s += input_img[x+i, y+j] * kernel[i, j]
        output[x, y] = s


# ---------------- ReLU ----------------
@cuda.jit
def relu(data):
    idx = cuda.grid(1)
    if idx < data.size:
        if data.flat[idx] < 0:
            data.flat[idx] = 0


# ---------------- MaxPooling 2x2 ----------------
@cuda.jit
def maxpool(input_img, output):
```

```
    x, y = cuda.grid(2)

    if x < output.shape[0] and y < output.shape[1]:
        in_x = x * 2
        in_y = y * 2

        max_val = input_img[in_x, in_y]

        for i in range(2):
            for j in range(2):
                val = input_img[in_x+i, in_y+j]
                if val > max_val:
                    max_val = val

        output[x, y] = max_val


# ---------------- Fully Connected ----------------
@cuda.jit
def fully_connected(input_vec, weights, bias, output):
    cls = cuda.grid(1)

    if cls < 2:
        s = bias[cls]
        for i in range(input_vec.size):
            s += input_vec[i] * weights[cls, i]
        output[cls] = s


# ---------------- Softmax ----------------
@cuda.jit
def softmax(input_vec, output):
    total = 0.0
    for i in range(2):
        total += math.exp(input_vec[i])

    for i in range(2):
        output[i] = math.exp(input_vec[i]) / total

kernel = np.random.randn(3,3).astype(np.float32)

conv_out_size = IMG_SIZE - 2
pool_size = conv_out_size // 2
fc_input_size = pool_size * pool_size

fc_weights = np.random.randn(2, fc_input_size).astype(np.float32)
fc_bias = np.zeros(2, dtype=np.float32)

EPOCHS = 5

threads2d = (16,16)
```

```
blocks2d = (math.ceil((IMG_SIZE-2)/16),
            math.ceil((IMG_SIZE-2)/16))

for epoch in range(EPOCHS):

    correct = 0

    for i in range(len(X)):

        img = X[i]
        label = y[i]

        # ----------- Conv -----------
        d_input = cuda.to_device(img)
        d_kernel = cuda.to_device(kernel)

        conv_out = np.zeros((IMG_SIZE-2, IMG_SIZE-2),
dtype=np.float32)
        d_conv = cuda.to_device(conv_out)

        conv2d[blocks2d, threads2d](d_input, d_kernel, d_conv)

        # ----------- ReLU -----------
        relu[256,256](d_conv)

        # ----------- Pool -----------
        pool_out = np.zeros((pool_size, pool_size), dtype=np.float32)
        d_pool = cuda.to_device(pool_out)

        maxpool[blocks2d, threads2d](d_conv, d_pool)

        flat = d_pool.copy_to_host().flatten()
        d_flat = cuda.to_device(flat)

        # ----------- FC -----------
        d_weights = cuda.to_device(fc_weights)
        d_bias = cuda.to_device(fc_bias)

        fc_out = np.zeros(2, dtype=np.float32)
        d_fc_out = cuda.to_device(fc_out)

        fully_connected[1,2](d_flat, d_weights, d_bias, d_fc_out)

        # ----------- Softmax -----------
        soft_out = np.zeros(2, dtype=np.float32)
        d_soft = cuda.to_device(soft_out)

        softmax[1,1](d_fc_out, d_soft)

        prediction = np.argmax(d_soft.copy_to_host())
```

```
        if prediction == label:
            correct += 1

    acc = correct / len(X)
    print(f"Epoch {epoch+1} Accuracy: {acc*100:.2f}%")
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:697: NumbaPerformanceWarning: Grid size 4 will likely
result in GPU under-utilization due to low occupancy.
  warn(NumbaPerformanceWarning(msg))
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatch
er.py:697: NumbaPerformanceWarning: Grid size 4 will likely result in
GPU under-utilization due to low occupancy.
  warn(NumbaPerformanceWarning(msg))
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatch
er.py:697: NumbaPerformanceWarning: Grid size 1 will likely result in
GPU under-utilization due to low occupancy.
  warn(NumbaPerformanceWarning(msg))
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatch
er.py:697: NumbaPerformanceWarning: Grid size 1 will likely result in
GPU under-utilization due to low occupancy.
  warn(NumbaPerformanceWarning(msg))
```

```
Epoch 1 Accuracy: 52.50%
Epoch 2 Accuracy: 52.50%
Epoch 3 Accuracy: 52.50%
Epoch 4 Accuracy: 52.50%
Epoch 5 Accuracy: 52.50%
```

```
def predict_image(img):

    # ----------- Conv -----------
    d_input = cuda.to_device(img)
    d_kernel = cuda.to_device(kernel)

    conv_out = np.zeros((IMG_SIZE-2, IMG_SIZE-2), dtype=np.float32)
    d_conv = cuda.to_device(conv_out)

    conv2d[blocks2d, threads2d](d_input, d_kernel, d_conv)

    # ----------- ReLU -----------
    relu[256,256](d_conv)

    # ----------- Pool -----------
    pool_out = np.zeros((pool_size, pool_size), dtype=np.float32)
    d_pool = cuda.to_device(pool_out)

    maxpool[blocks2d, threads2d](d_conv, d_pool)

    flat = d_pool.copy_to_host().flatten()
    d_flat = cuda.to_device(flat)
```

```
    # ----------- FC -----------
    d_weights = cuda.to_device(fc_weights)
    d_bias = cuda.to_device(fc_bias)

    fc_out = np.zeros(2, dtype=np.float32)
    d_fc_out = cuda.to_device(fc_out)

    fully_connected[1,2](d_flat, d_weights, d_bias, d_fc_out)

    # ----------- Softmax -----------
    soft_out = np.zeros(2, dtype=np.float32)
    d_soft = cuda.to_device(soft_out)

    softmax[1,1](d_fc_out, d_soft)

    prediction = np.argmax(d_soft.copy_to_host())

    return prediction

class_names = ["Mask", "No Mask"]

n = len(X)
cols = 5
rows = math.ceil(n / cols)

plt.figure(figsize=(12,8))

for i in range(n):

    pred = predict_image(X[i])
    actual = y[i]

    plt.subplot(rows, cols, i+1)
    plt.imshow(X[i], cmap='gray')
    plt.axis("off")

    color = "green" if pred == actual else "red"

    plt.title(f"P:{class_names[pred]}\nA:{class_names[actual]}",
color=color)

plt.tight_layout()
plt.show()
```

For multiclass from scratch,multiclass ,Mnist dataset,use 5 layers,hyperparameter, epochs atleast 50

For multiclass from scratch,multiclass ,Mnist dataset,use 5 layers,hyperparameter, epochs atleast 50

```
!pip install numba opencv-python matplotlib
```

```
Requirement already satisfied: numba in
/usr/local/lib/python3.12/dist-packages (0.60.0)
Requirement already satisfied: opencv-python in
/usr/local/lib/python3.12/dist-packages (4.13.0.92)
Requirement already satisfied: matplotlib in
/usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in
/usr/local/lib/python3.12/dist-packages (from numba) (0.43.0)
Requirement already satisfied: numpy<2.1,>=1.22 in
/usr/local/lib/python3.12/dist-packages (from numba) (2.0.2)
Requirement already satisfied: contourpy>=1.0.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (26.0)
Requirement already satisfied: pillow>=8 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in
/usr/local/lib/python3.12/dist-packages (from matplotlib) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in
/usr/local/lib/python3.12/dist-packages (from matplotlib)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in
/usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7-
>matplotlib) (1.17.0)
```

```
import numpy as np
import cv2
import os
import math
from numba import cuda
import matplotlib.pyplot as plt

print("CUDA Available:", cuda.is_available())
```

```
CUDA Available: True
```

```
EPOCHS = 50
LR = 0.01
BATCH_SIZE = 64
```

```
CONV1_FILTERS = 8
CONV2_FILTERS = 16
KERNEL_SIZE = 3
HIDDEN1 = 128
HIDDEN2 = 64
NUM_CLASSES = 10

import numpy as np
from numba import cuda

@cuda.jit
def conv2d(input_img, kernel, output):
    i, j, k = cuda.grid(3)

    if (i < output.shape[0] and
        j < output.shape[1] and
        k < output.shape[2]):

        val = 0.0
        for ki in range(kernel.shape[1]):
            for kj in range(kernel.shape[2]):
                val += input_img[i+ki, j+kj] * kernel[k, ki, kj]

        output[i, j, k] = val

def relu(x):
    return np.maximum(0, x)

def softmax(x):
    exp = np.exp(x - np.max(x))
    return exp / np.sum(exp)

def cross_entropy(pred, label):
    return -np.log(pred[label] + 1e-9)

class Dense:
    def __init__(self, inp, out):
        self.W = np.random.randn(inp, out) * 0.01
        self.b = np.zeros(out)

    def forward(self, x):
        self.x = x
        return x @ self.W + self.b

    def backward(self, grad, lr):
        dW = np.outer(self.x, grad)
        db = grad
        dx = self.W @ grad

        self.W -= lr * dW
```

```
        self.b -= lr * db
        return dx

class ConvLayer:
    def __init__(self, in_channels, out_channels, k):
        self.kernels = np.random.randn(out_channels, k, k) * 0.01

    def forward(self, x):
        h, w = x.shape
        k = self.kernels.shape[1]
        out_h = h - k + 1
        out_w = w - k + 1

        output = np.zeros((out_h, out_w, self.kernels.shape[0]))

        d_x = cuda.to_device(x.astype(np.float32))
        d_k = cuda.to_device(self.kernels.astype(np.float32))
        d_out = cuda.to_device(output.astype(np.float32))

        threads = (8, 8, 1)
        blocks = (
            (out_h+7)//8,
            (out_w+7)//8,
            self.kernels.shape[0]
        )

        conv2d[blocks, threads](d_x, d_k, d_out)
        return d_out.copy_to_host()

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
import numpy as np

mnist = fetch_openml('mnist_784', version=1)

# convert DataFrame → numpy
X = mnist.data.to_numpy().reshape(-1, 28, 28).astype(np.float32) / 255
y = mnist.target.astype(int).to_numpy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42
)

conv1 = ConvLayer(1, CONV1_FILTERS, KERNEL_SIZE)
conv2 = ConvLayer(CONV1_FILTERS, CONV2_FILTERS, KERNEL_SIZE)

fc1 = Dense(24*24*CONV2_FILTERS, HIDDEN1)
fc2 = Dense(HIDDEN1, HIDDEN2)
fc3 = Dense(HIDDEN2, NUM_CLASSES)
```

```
for epoch in range(EPOCHS):
    loss_total = 0

    for img, label in zip(X_train, y_train):

        # ----- Forward -----
        x = conv1.forward(img)
        x = relu(x)

        x = conv2.forward(x[:,:,0])
        x = relu(x)

        x = x.reshape(-1)

        x = relu(fc1.forward(x))
        x = relu(fc2.forward(x))
        logits = fc3.forward(x)

        probs = softmax(logits)
        loss = cross_entropy(probs, label)
        loss_total += loss

        # ----- Backprop -----
        grad = probs
        grad[label] -= 1

        grad = fc3.backward(grad, LR)
        grad = fc2.backward(grad, LR)
        grad = fc1.backward(grad, LR)

    print(f"Epoch {epoch+1}/{EPOCHS}, Loss
{loss_total/len(X_train):.4f}")
```

```
Epoch 1/50, Loss 2.3033
Epoch 2/50, Loss 2.3033
Epoch 3/50, Loss 2.3033
Epoch 4/50, Loss 2.3033
Epoch 5/50, Loss 2.3033
```

```
---------------------------------------------------------------------
-----
KeyboardInterrupt                         Traceback (most recent call
last)
/tmp/ipython-input-356/3719626808.py in <cell line: 0>()
     27         grad = fc3.backward(grad, LR)
     28         grad = fc2.backward(grad, LR)
---> 29         grad = fc1.backward(grad, LR)
     30
     31     print(f"Epoch {epoch+1}/{EPOCHS}, Loss
{loss_total/len(X_train):.4f}")
```

```
/tmp/ipython-input-356/2346969945.py in backward(self, grad, lr)
      9
     10     def backward(self, grad, lr):
---> 11         dW = np.outer(self.x, grad)
     12         db = grad
     13         dx = self.W @ grad

/usr/local/lib/python3.12/dist-packages/numpy/_core/numeric.py in
outer(a, b, out)
    969     a = asarray(a)
    970     b = asarray(b)
--> 971     return multiply(a.ravel()[:, newaxis], b.ravel()
[newaxis, :], out)
    972
    973

KeyboardInterrupt:
```

```python
def predict(img):
    x = conv1.forward(img)
    x = relu(x)

    x = conv2.forward(x[:,:,0])
    x = relu(x)

    x = x.reshape(-1)
    x = relu(fc1.forward(x))
    x = relu(fc2.forward(x))
    x = fc3.forward(x)

    return np.argmax(softmax(x))
```

```
import numpy as np
from numba import cuda


# CUDA Kernel for one step of Bitonic Sort
@cuda.jit
def bitonic_sort_step(d_values, j, k):
    idx = cuda.grid(1)

    if idx < d_values.size:
        ixj = idx ^ j

        if ixj > idx:
            # Ascending sort
            if (idx & k) == 0:
                if d_values[idx] > d_values[ixj]:
                    d_values[idx], d_values[ixj] = d_values[ixj],
d_values[idx]
            # Descending sort
            else:
                if d_values[idx] < d_values[ixj]:
                    d_values[idx], d_values[ixj] = d_values[ixj],
d_values[idx]


def bitonic_sort_gpu(arr):
    n = arr.size

    # Copy array to GPU
    d_arr = cuda.to_device(arr)

    threads_per_block = 256
    blocks_per_grid = (n + threads_per_block - 1) // threads_per_block

    k = 2
    while k <= n:
        j = k // 2
        while j > 0:
            bitonic_sort_step[blocks_per_grid, threads_per_block]
(d_arr, j, k)
            cuda.synchronize()
            j //= 2
        k *= 2

    # Copy result back to CPU
    return d_arr.copy_to_host()
```

```python
# Main Execution
if __name__ == "__main__":
    arr = np.array([3, 7, 4, 8, 6, 2, 1, 5], dtype=np.int32)

    print("Original Array:")
    print(arr)

    sorted_arr = bitonic_sort_gpu(arr)

    print("\nSorted Array:")
    print(sorted_arr)
```

```
Original Array:
[3 7 4 8 6 2 1 5]

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))


Sorted Array:
[1 2 3 4 5 6 7 8]
```

```python
# mnist --> 5 layers(conv and pooling) --> 50 epochs

import numpy as np
from numba import cuda
import math

import torchvision
import torchvision.transforms as transforms

# Download MNIST
transform = transforms.ToTensor()

train_dataset = torchvision.datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_dataset = torchvision.datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

print("MNIST downloaded successfully")
```

```
100%|██████████| 9.91M/9.91M [00:01<00:00, 5.05MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 134kB/s]
100%|██████████| 1.65M/1.65M [00:01<00:00, 1.27MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 3.32MB/s]

MNIST downloaded successfully
```

```python
image, label = train_dataset[0]

# Convert to numpy
image = image.squeeze().numpy().astype("float32")

print("Label:", label)
print("Shape:", image.shape)
```

```
Label: 5
Shape: (28, 28)
```

```python
@cuda.jit
def conv2d(input_img, kernel, output):
    h, w = cuda.grid(2)
```

```
    H = input_img.shape[0]
    W = input_img.shape[1]

    if h < H and w < W:
        sum_val = 0.0

        for kh in range(3):
            for kw in range(3):
                ih = h + kh - 1
                iw = w + kw - 1

                if 0 <= ih < H and 0 <= iw < W:
                    sum_val += input_img[ih, iw] * kernel[kh, kw]

        output[h, w] = sum_val

@cuda.jit
def relu(x):
    i, j = cuda.grid(2)

    if i < x.shape[0] and j < x.shape[1]:
        if x[i, j] < 0:
            x[i, j] = 0

@cuda.jit
def maxpool_kernel(input_img, output):
    h, w = cuda.grid(2)

    H_out = output.shape[0]
    W_out = output.shape[1]

    if h < H_out and w < W_out:
        max_val = -1e9

        for i in range(2):
            for j in range(2):
                val = input_img[h*2 + i, w*2 + j]
                if val > max_val:
                    max_val = val

        output[h, w] = max_val

d_img = cuda.to_device(image)

kernels = []
for _ in range(5):
    k = np.random.randn(3,3).astype(np.float32) * 0.1
    kernels.append(cuda.to_device(k))

def launch_2d(kernel_func, input_arr, *args):
    H, W = input_arr.shape
```

```
    threads = (16,16)
    blocks = (math.ceil(H/16), math.ceil(W/16))
    kernel_func[blocks, threads](input_arr, *args)

# conv1
d_out1 = cuda.device_array((28,28), dtype=np.float32)
launch_2d(conv2d, d_img, kernels[0], d_out1)
launch_2d(relu, d_out1)
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 4 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatch
er.py:696: NumbaPerformanceWarning: Grid size 4 will likely result in
GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
```

```
d_out2 = cuda.device_array((28,28), dtype=np.float32)
launch_2d(conv2d, d_out1, kernels[1], d_out2)
launch_2d(relu, d_out2)

d_pool1 = cuda.device_array((14,14), dtype=np.float32)
threads = (16,16)
blocks = (math.ceil(14/16), math.ceil(14/16))
maxpool_kernel[blocks, threads](d_out2, d_pool1)
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
```

```
d_out3 = cuda.device_array((14,14), dtype=np.float32)
launch_2d(conv2d, d_pool1, kernels[2], d_out3)
launch_2d(relu, d_out3)
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
```

```
d_out4 = cuda.device_array((14,14), dtype=np.float32)
launch_2d(conv2d, d_out3, kernels[3], d_out4)
launch_2d(relu, d_out4)

d_pool2 = cuda.device_array((7,7), dtype=np.float32)
threads = (16,16)
blocks = (math.ceil(7/16), math.ceil(7/16))
maxpool_kernel[blocks, threads](d_out4, d_pool2)
```

```
d_out5 = cuda.device_array((7,7), dtype=np.float32)
launch_2d(conv2d, d_pool2, kernels[4], d_out5)
launch_2d(relu, d_out5)

final_output = d_out5.copy_to_host()
flattened = final_output.flatten()

print("Final feature size:", flattened.shape)
```

```
Final feature size: (49,)
```

```
# fc_weights = np.random.randn(49, 10).astype(np.float32) * 0.01
fc_weights = np.random.randn(784, 10).astype(np.float32) * 0.01
fc_bias = np.zeros(10, dtype=np.float32)

d_fc_weights = cuda.to_device(fc_weights)
d_fc_bias = cuda.to_device(fc_bias)

@cuda.jit
def fc_forward(x, weights, bias, output):
    i = cuda.grid(1)
    if i < 10:
        s = 0.0
        for j in range(x.shape[0]):
            s += x[j] * weights[j, i]
        output[i] = s + bias[i]

@cuda.jit
def fc_forward(x, weights, bias, output):
    i = cuda.grid(1)
    if i < 10:
        s = 0.0
        for j in range(x.shape[0]):
            s += x[j] * weights[j, i]
        output[i] = s + bias[i]

def softmax(x):
    e = np.exp(x - np.max(x))
    return e / np.sum(e)

def cross_entropy(pred, label):
    return -np.log(pred[label] + 1e-8)

def fc_backward(x, pred, label, weights):
    grad_out = pred.copy()
    grad_out[label] -= 1.0   # dL/dz

    grad_w = np.outer(x, grad_out)
    grad_b = grad_out
```

```
    grad_input = np.dot(weights, grad_out)

    return grad_w, grad_b, grad_input

learning_rate = 0.01

# fc_weights -= learning_rate * grad_w
# fc_bias -= learning_rate * grad_b

# d_fc_weights = cuda.to_device(fc_weights)
# d_fc_bias = cuda.to_device(fc_bias)

@cuda.jit
def conv2d_input_backward(grad_output, kernel, grad_input):
    h, w = cuda.grid(2)

    H = grad_input.shape[0]
    W = grad_input.shape[1]

    if h < H and w < W:
        s = 0.0

        for kh in range(3):
            for kw in range(3):
                oh = h - kh + 1
                ow = w - kw + 1

                if 0 <= oh < H and 0 <= ow < W:
                    s += grad_output[oh, ow] * kernel[kh, kw]

        grad_input[h, w] = s

@cuda.jit
def conv2d_backward(input_img, grad_output, grad_kernel):
    kh, kw = cuda.grid(2)

    if kh < 3 and kw < 3:
        s = 0.0
        H = input_img.shape[0]
        W = input_img.shape[1]

        for h in range(H):
            for w in range(W):
                ih = h + kh - 1
                iw = w + kw - 1

                if 0 <= ih < H and 0 <= iw < W:
                    s += input_img[ih, iw] * grad_output[h, w]

        grad_kernel[kh, kw] = s
```

```
@cuda.jit
def relu_backward(grad_output, activation, grad_input):
    h, w = cuda.grid(2)

    if h < activation.shape[0] and w < activation.shape[1]:
        if activation[h, w] > 0:
            grad_input[h, w] = grad_output[h, w]
        else:
            grad_input[h, w] = 0.0

epochs = 5

for epoch in range(epochs):
    total_loss = 0

    # for idx in range(len(train_dataset)):
    for idx in range(2000):

        image, label = train_dataset[idx]
        image = image.squeeze().numpy().astype(np.float32)

        # ---- Move to GPU ----
        d_img = cuda.to_device(image)

        # ---- Forward Conv Layers ----
        activations = []
        inputs = []

        current = d_img

        for k in range(5):

            inputs.append(current.copy_to_host())

            d_kernel = cuda.to_device(kernels[k])
            d_out = cuda.device_array(current.shape, dtype=np.float32)

            threads = (16,16)
            blocks = (math.ceil(current.shape[0]/16),
                      math.ceil(current.shape[1]/16))

            conv2d[blocks, threads](current, d_kernel, d_out)
            relu[blocks, threads](d_out)

            current = d_out
            activations.append(current.copy_to_host())

        # ---- Flatten ----
        final_feature = current.copy_to_host().flatten()

        # ---- FC Forward ----
```

```
        logits = np.dot(final_feature, fc_weights) + fc_bias
        pred = softmax(logits)

        loss = cross_entropy(pred, label)
        total_loss += loss

        # ---- FC Backward ----
        grad_w, grad_b, grad_input = fc_backward(
            final_feature, pred, label, fc_weights
        )

        # Update FC
        fc_weights -= learning_rate * grad_w
        fc_bias -= learning_rate * grad_b

        # ---- Backprop through Conv5 → Conv1 ----
        grad_input = grad_input.reshape(28,28)

        for k in reversed(range(5)):

            d_grad_output = cuda.to_device(grad_input)
            d_input = cuda.to_device(inputs[k])
            d_grad_kernel = cuda.device_array((3,3), dtype=np.float32)

            threads = (16,16)
            blocks = (1,1)

            conv2d_backward[blocks, threads](
                d_input, d_grad_output, d_grad_kernel
            )

            grad_kernel = d_grad_kernel.copy_to_host()

            kernels[k] -= learning_rate * grad_kernel

            d_grad_input = cuda.device_array(inputs[k].shape,
dtype=np.float32)

            threads = (16,16)
            blocks = (math.ceil(inputs[k].shape[0]/16),
                      math.ceil(inputs[k].shape[1]/16))

            d_kernel_current = cuda.to_device(kernels[k])

            conv2d_input_backward[blocks, threads](
                d_grad_output, d_kernel_current, d_grad_input
            )

            grad_input = d_grad_input.copy_to_host()

    print("Epoch:", epoch+1, "Loss:", total_loss/len(train_dataset))
```

```
Epoch: 1 Loss: 0.07677122
Epoch: 2 Loss: 0.07677566
Epoch: 3 Loss: 0.07677639
Epoch: 4 Loss: 0.07677654
Epoch: 5 Loss: 0.07677651
```

```python
def predict(image):
    image = image.squeeze().numpy().astype(np.float32)
    d_img = cuda.to_device(image)

    current = d_img

    # 5 Conv Layers
    for k in range(5):
        d_kernel = cuda.to_device(kernels[k])
        d_out = cuda.device_array(current.shape, dtype=np.float32)

        threads = (16,16)
        blocks = (math.ceil(current.shape[0]/16),
                  math.ceil(current.shape[1]/16))

        conv2d[blocks, threads](current, d_kernel, d_out)
        relu[blocks, threads](d_out)

        current = d_out

    final_feature = current.copy_to_host().flatten()

    logits = np.dot(final_feature, fc_weights) + fc_bias
    pred = softmax(logits)

    return np.argmax(pred)

image, label = test_dataset[0]

prediction = predict(image)

print("True Label:", label)
print("Predicted :", prediction)
```

```
True Label: 7
Predicted : 7
```

```
import seaborn as sns
import pandas as pd

# Load Titanic dataset from seaborn
df = sns.load_dataset('titanic')

df.head()
```

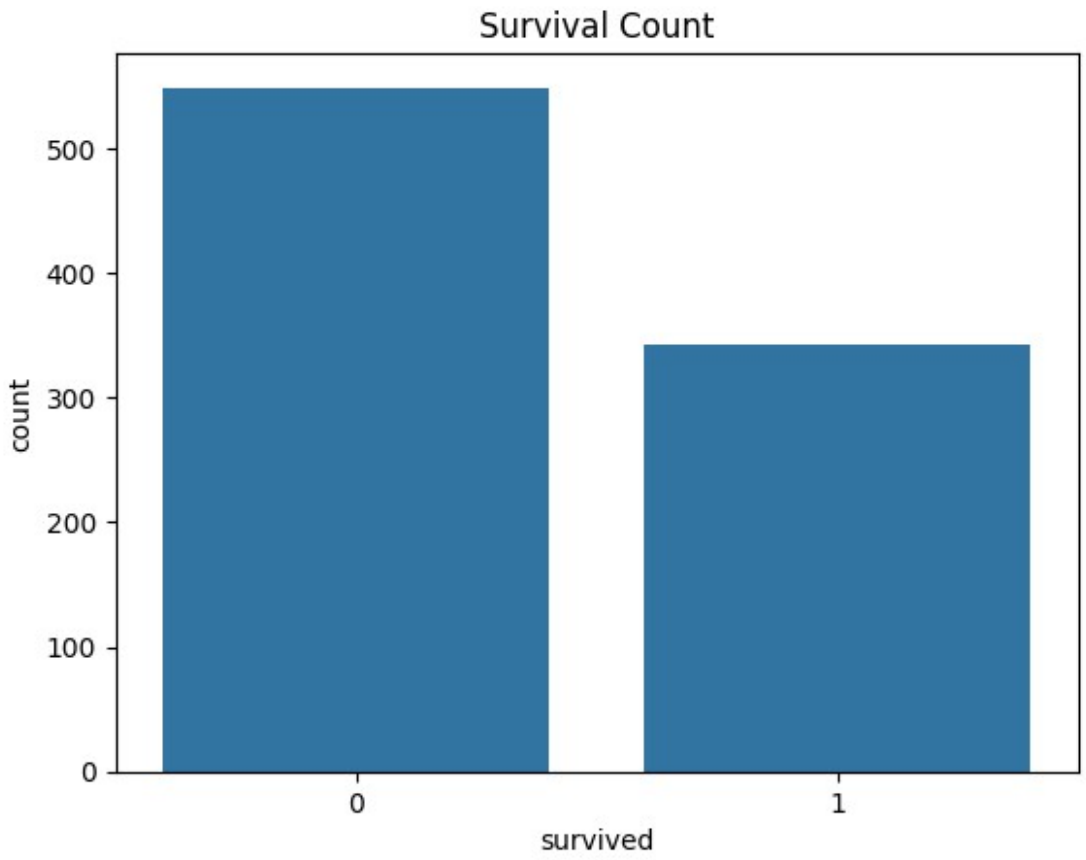
```
   survived  pclass     sex   age  sibsp  parch     fare embarked
class  \
0         0       3    male  22.0      1      0   7.2500        S
Third
1         1       1  female  38.0      1      0  71.2833        C
First
2         1       3  female  26.0      0      0   7.9250        S
Third
3         1       1  female  35.0      1      0  53.1000        S
First
4         0       3    male  35.0      0      0   8.0500        S
Third

     who  adult_male deck  embark_town alive  alone
0    man        True  NaN  Southampton    no  False
1  woman       False    C    Cherbourg   yes  False
2  woman       False  NaN  Southampton   yes   True
3  woman       False    C  Southampton   yes  False
4    man        True  NaN  Southampton    no   True
```

```
print(df.info())
print(df.isnull().sum())
```

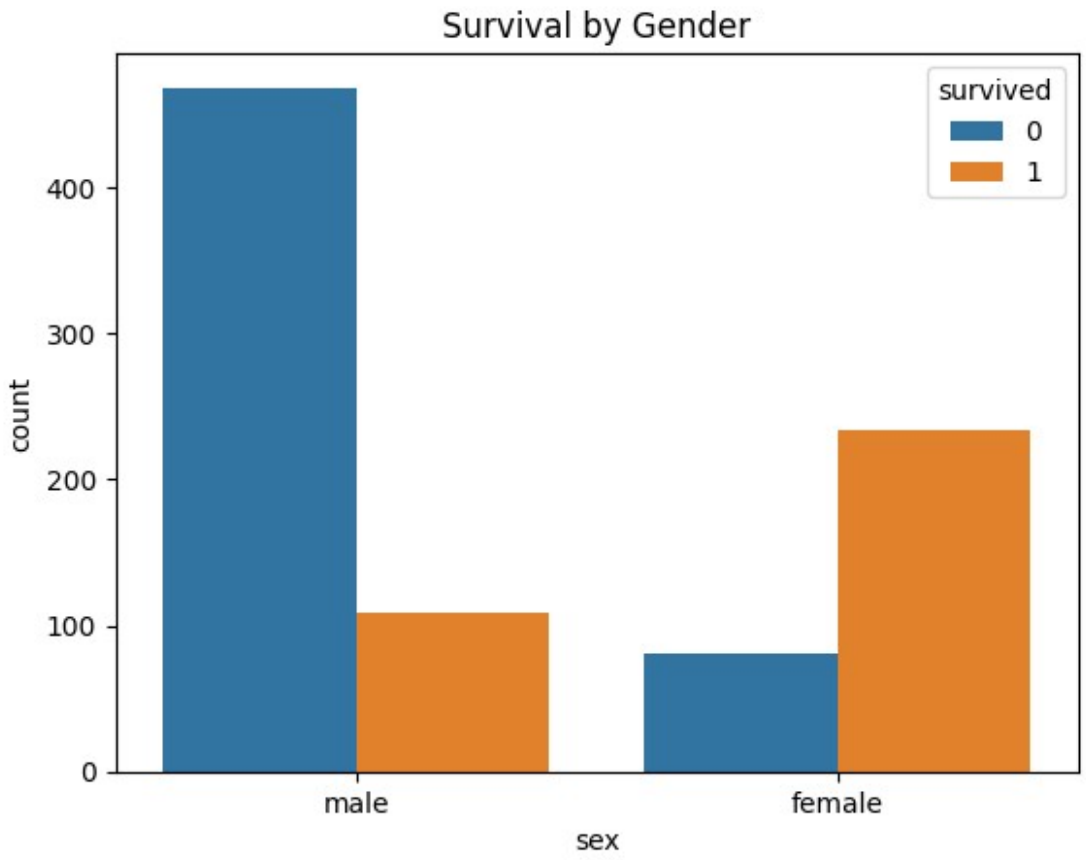
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
```

```
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
None
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

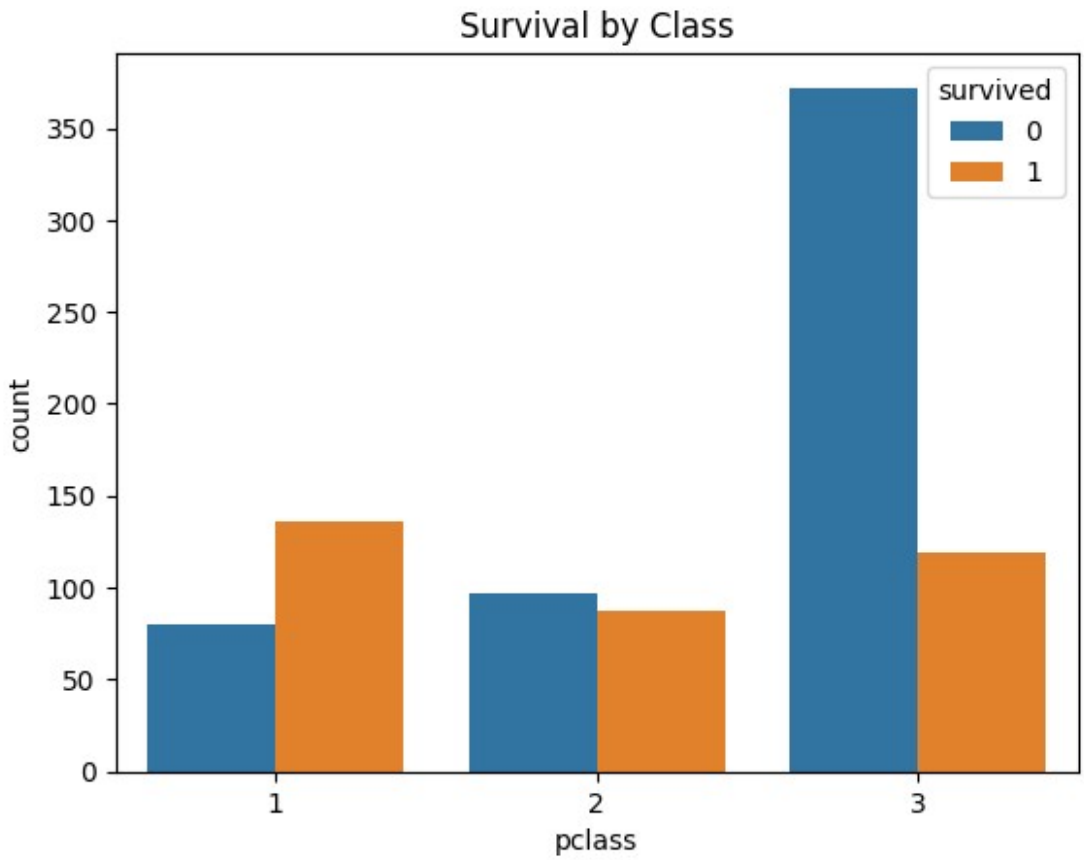
```python
import matplotlib.pyplot as plt

sns.countplot(x='survived', data=df)
plt.title("Survival Count")
plt.show()
```



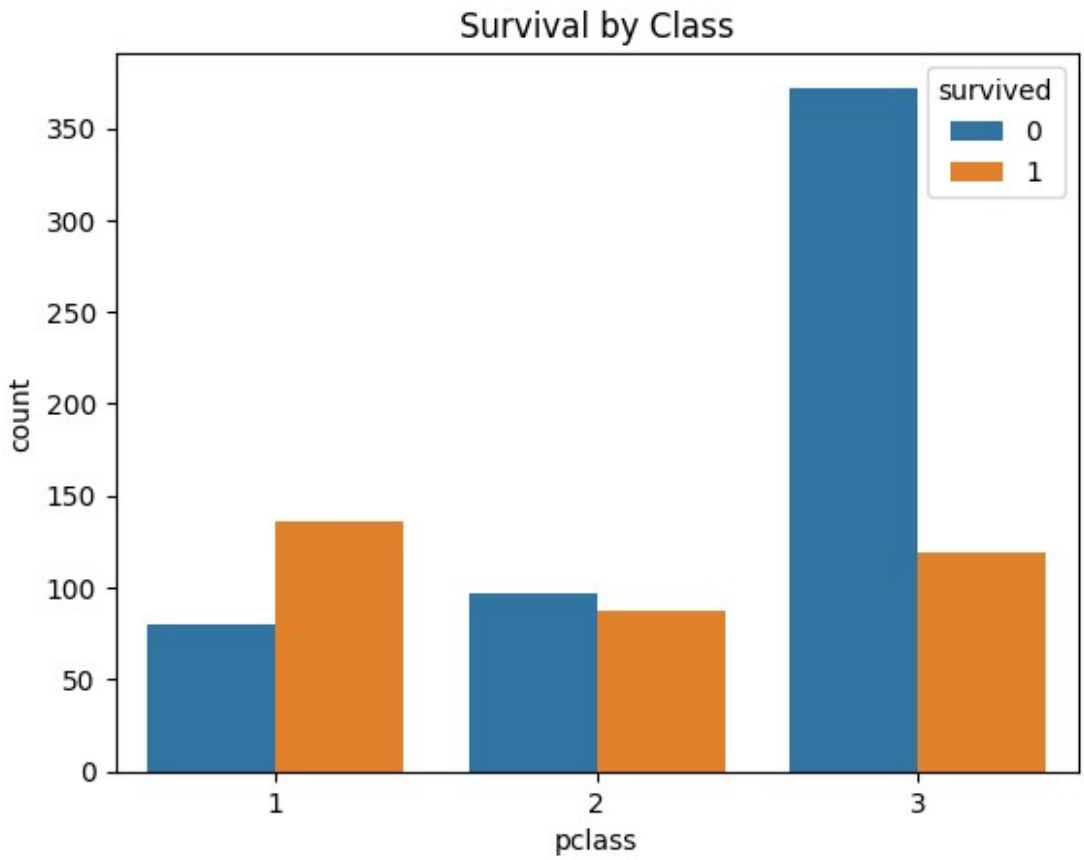

```
sns.countplot(x='sex', hue='survived', data=df)
plt.title("Survival by Gender")
plt.show()
```



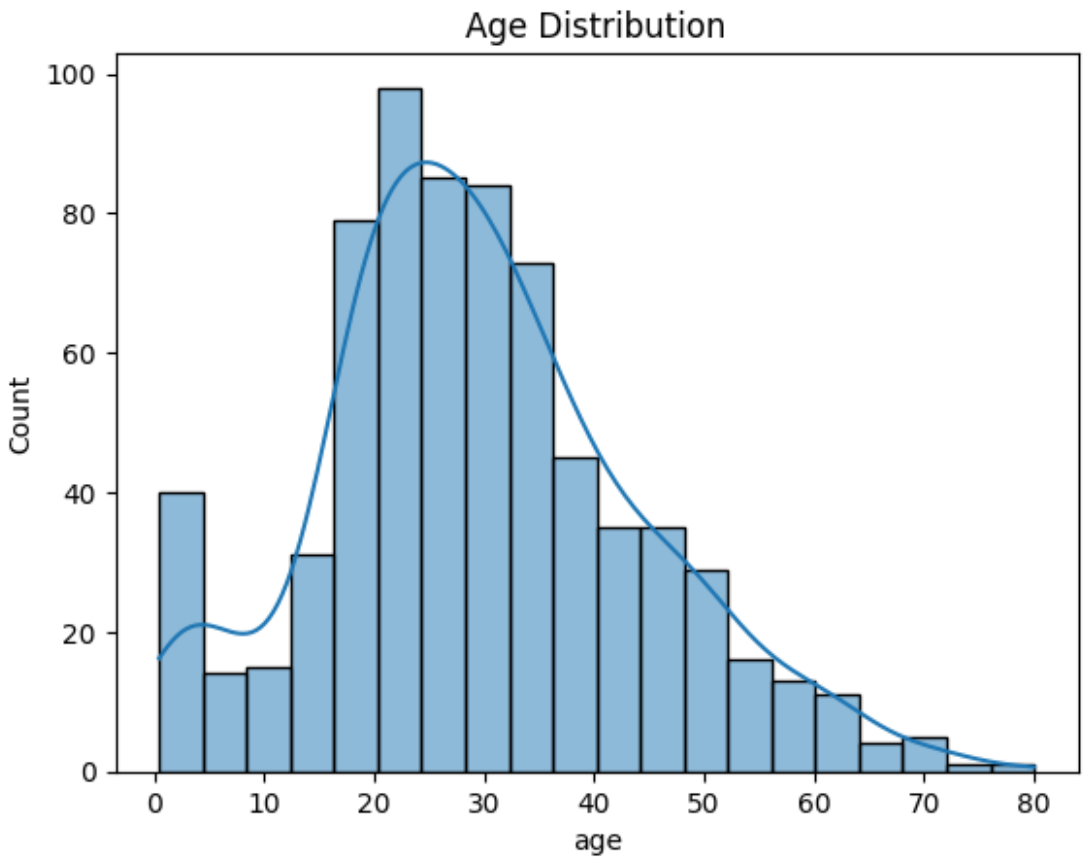

```
sns.countplot(x='pclass', hue='survived', data=df)
plt.title("Survival by Class")
plt.show()
```



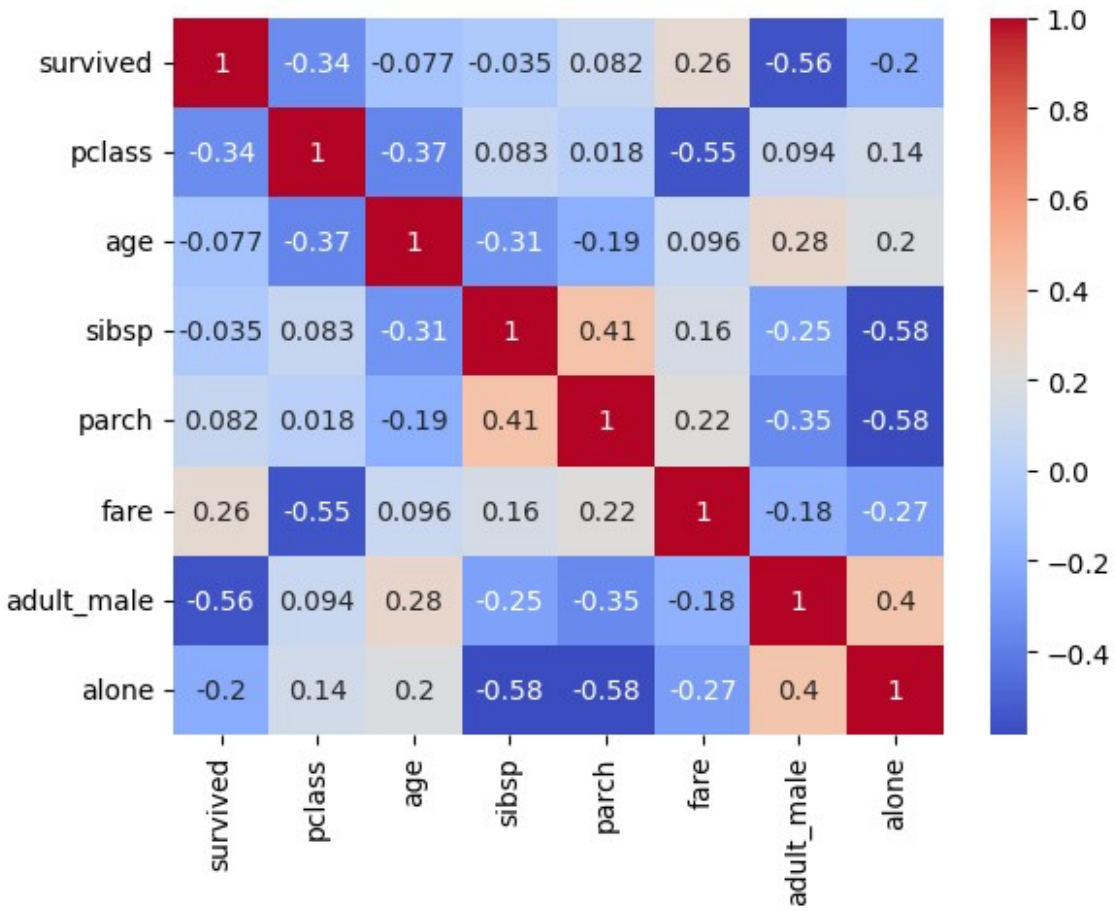

```
sns.countplot(x='pclass', hue='survived', data=df)
plt.title("Survival by Class")
plt.show()
```




```
sns.histplot(df['age'].dropna(), kde=True)
plt.title("Age Distribution")
plt.show()
```




```
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap='coolwarm')
plt.show()
```




```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_cols = ['sex', 'embarked', 'who']

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

X = df.drop('survived', axis=1)
y = df['survived']

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score

lr = LogisticRegression(max_iter=200)
```

```
lr.fit(X_train, y_train)

pred_lr = lr.predict(X_test)
print("Logistic Accuracy:", accuracy_score(y_test, pred_lr))
```

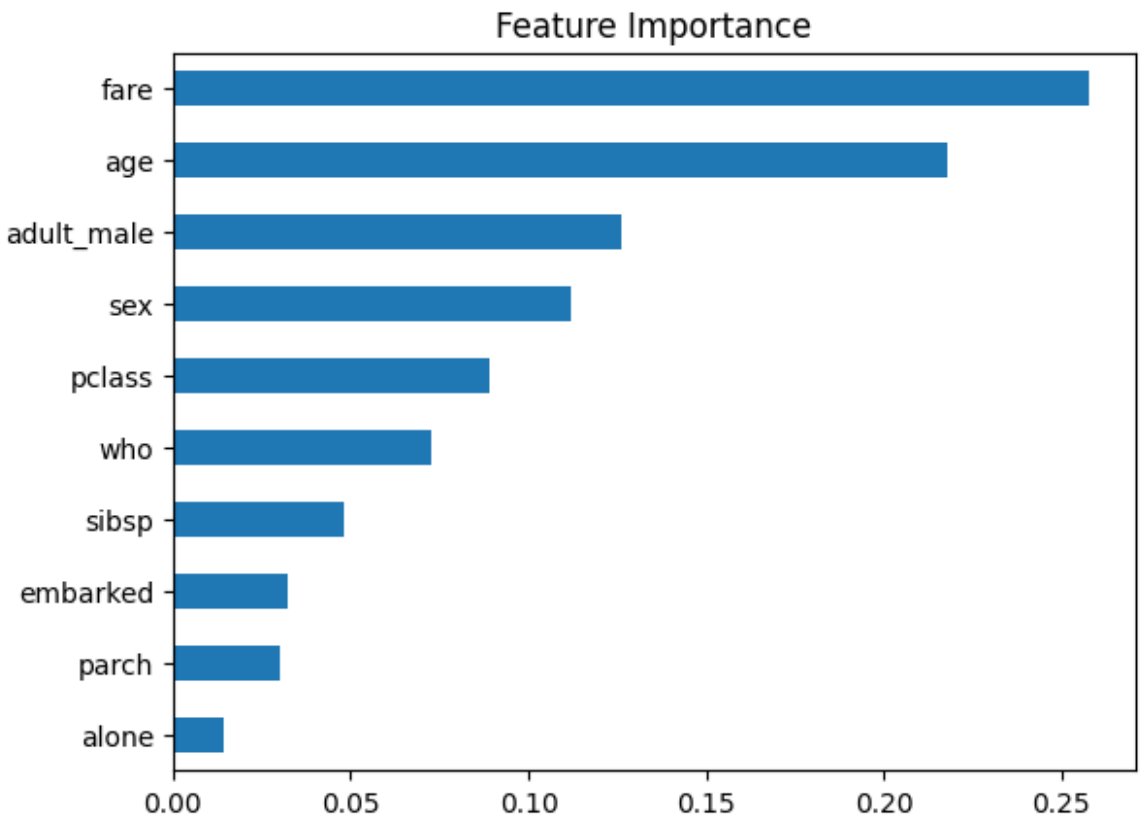
```
Logistic Accuracy: 0.8156424581005587
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/
_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
(status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as
shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-
regression
  n_iter_i = _check_optimize_result(
```

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=100)
rf.fit(X_train, y_train)

pred_rf = rf.predict(X_test)
print("Random Forest Accuracy:", accuracy_score(y_test, pred_rf))
```

```
Random Forest Accuracy: 0.8212290502793296
```

```
import matplotlib.pyplot as plt
import pandas as pd

importances = rf.feature_importances_
feat_names = X.columns

feat_imp = pd.Series(importances, index=feat_names).sort_values()

feat_imp.plot(kind='barh')
plt.title("Feature Importance")
plt.show()
```

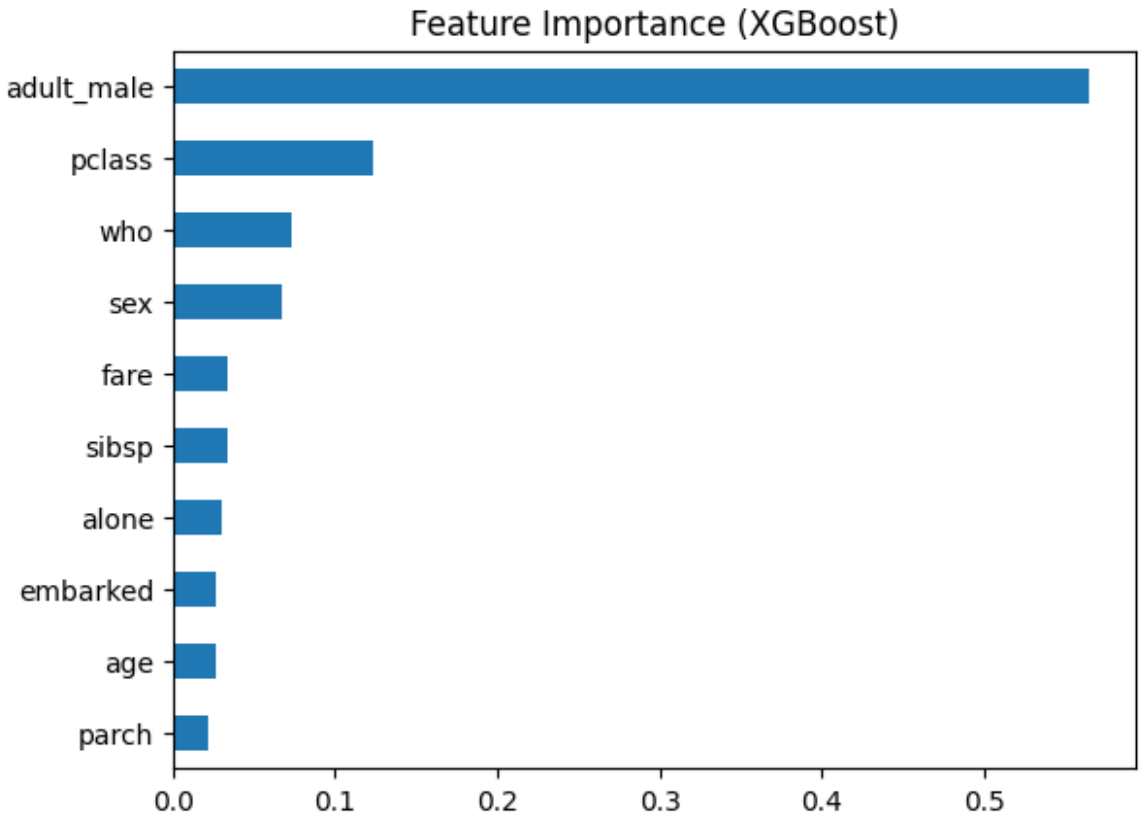



```
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score

model = XGBClassifier(
    n_estimators=200,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42
)

model.fit(X_train, y_train)

preds = model.predict(X_test)

print("XGBoost Accuracy:", accuracy_score(y_test, preds))
```

```
XGBoost Accuracy: 0.8212290502793296
```

```
import matplotlib.pyplot as plt

importances = model.feature_importances_

pd.Series(importances,
```

```
index=X.columns).sort_values().plot(kind='barh')
plt.title("Feature Importance (XGBoost)")
plt.show()
```



```
!pip install numba
```

```
Requirement already satisfied: numba in
/usr/local/lib/python3.12/dist-packages (0.60.0)
Requirement already satisfied: llvmlite<0.44,>=0.43.0dev0 in
/usr/local/lib/python3.12/dist-packages (from numba) (0.43.0)
Requirement already satisfied: numpy<2.1,>=1.22 in
/usr/local/lib/python3.12/dist-packages (from numba) (2.0.2)
```

Normal Case

```
import numpy as np
from numba import cuda

@cuda.jit
def reduce_kernel(input_arr, output_arr):
    # Shared memory
    sdata = cuda.shared.array(1024, dtype=numba.int32)

    tid = cuda.threadIdx.x
    i = cuda.blockIdx.x * cuda.blockDim.x + tid

    # Load data into shared memory
    if i < input_arr.size:
        sdata[tid] = input_arr[i]
    else:
        sdata[tid] = 0
    cuda.syncthreads()

    # Reduction (tree pattern)
    stride = 1
    while stride < cuda.blockDim.x:
        if tid % (2 * stride) == 0:
            sdata[tid] += sdata[tid + stride]
        stride *= 2
        cuda.syncthreads()

    # Write result
    if tid == 0:
        output_arr[cuda.blockIdx.x] = sdata[0]

import numba

# Input
arr = np.array([1,2,3,4,5,6,7,8], dtype=np.int32)

# Device memory
d_input = cuda.to_device(arr)
d_output = cuda.device_array(1, dtype=np.int32)
```

```
# Launch kernel
threads_per_block = 8
blocks = 1

reduce_kernel[blocks, threads_per_block](d_input, d_output)

# Copy result back
result = d_output.copy_to_host()

print("Final Sum:", result[0])
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))

Final Sum: 36
```

Handle odd number case and take input from user

```
import numpy as np
import numba
from numba import cuda

@cuda.jit
def reduce_kernel(input_arr, output_arr, n):
    # Dynamic shared memory
    sdata = cuda.shared.array(shape=0, dtype=numba.int32)

    tid = cuda.threadIdx.x
    i = cuda.blockIdx.x * cuda.blockDim.x + tid

    # Load elements safely
    if i < n:
        sdata[tid] = input_arr[i]
    else:
        sdata[tid] = 0
    cuda.syncthreads()

    # Tree-based reduction
    stride = 1
    while stride < cuda.blockDim.x:
        index = 2 * stride * tid
        if index < cuda.blockDim.x and index + stride < n:
            sdata[index] += sdata[index + stride]
        stride *= 2
        cuda.syncthreads()

    # Write block result
```

```
    if tid == 0:
        output_arr[cuda.blockIdx.x] = sdata[0]

# Take input from user
user_input = input("Enter numbers separated by space: ")
arr = np.array(list(map(int, user_input.split())), dtype=np.int32)

n = len(arr)

# GPU memory
d_input = cuda.to_device(arr)

threads_per_block = 1
while threads_per_block < n:
    threads_per_block *= 2   # next power of 2

blocks = 1

d_output = cuda.device_array(blocks, dtype=np.int32)

# Launch kernel (dynamic shared memory)
reduce_kernel[blocks, threads_per_block, 0, threads_per_block * 4]
(d_input, d_output, n)

# Get result
result = d_output.copy_to_host()

print("Final Sum:", result[0])
```

```
Enter numbers separated by space: 2 34 5 67 5 43 2 1
Final Sum: 159

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
```

Take numbers randomly with user giving number of values in input

```
import numpy as np
import numba
from numba import cuda
import random

@cuda.jit
def reduce_kernel(input_arr, output_arr, n):
    sdata = cuda.shared.array(shape=0, dtype=numba.int32)

    tid = cuda.threadIdx.x
```

```
    i = cuda.blockIdx.x * cuda.blockDim.x + tid

    # Load into shared memory safely
    if i < n:
        sdata[tid] = input_arr[i]
    else:
        sdata[tid] = 0
    cuda.syncthreads()

    # Reduction (tree pattern)
    stride = 1
    while stride < cuda.blockDim.x:
        index = 2 * stride * tid
        if index < cuda.blockDim.x and index + stride <
cuda.blockDim.x:
            sdata[index] += sdata[index + stride]
        stride *= 2
        cuda.syncthreads()

    # Write result
    if tid == 0:
        output_arr[cuda.blockIdx.x] = sdata[0]


#  User input (number of elements)
n = int(input("Enter number of elements: "))

# Generate random numbers
arr = np.array([random.randint(1, 10) for _ in range(n)],
dtype=np.int32)

print("Generated array:", arr)

# GPU memory
d_input = cuda.to_device(arr)

# Next power of 2 for threads
threads_per_block = 1
while threads_per_block < n:
    threads_per_block *= 2

blocks = 1
d_output = cuda.device_array(blocks, dtype=np.int32)

# Launch kernel
reduce_kernel[blocks, threads_per_block, 0, threads_per_block * 4]
(d_input, d_output, n)

# Result
result = d_output.copy_to_host()
```

```
print("Final Sum:", result[0])
```

```
Enter number of elements: 1024
Generated array: [ 9  1 10 ...  2  6  3]
Final Sum: 5669

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
```

Multibox reduction

```
import numpy as np
import numba
from numba import cuda
import random
import math

@cuda.jit
def reduce_kernel(input_arr, output_arr, n):
    sdata = cuda.shared.array(shape=0, dtype=numba.int32)

    tid = cuda.threadIdx.x
    i = cuda.blockIdx.x * cuda.blockDim.x + tid

    # Load into shared memory
    if i < n:
        sdata[tid] = input_arr[i]
    else:
        sdata[tid] = 0
    cuda.syncthreads()

    # Reduction within block
    stride = cuda.blockDim.x // 2
    while stride > 0:
        if tid < stride:
            sdata[tid] += sdata[tid + stride]
        stride //= 2
        cuda.syncthreads()

    # Write result of block
    if tid == 0:
        output_arr[cuda.blockIdx.x] = sdata[0]


# User input
n = int(input("Enter number of elements: "))

# Random input
```

```
arr = np.array([random.randint(1, 10) for _ in range(n)],
dtype=np.int32)

print("Generated array (first 20 elements):", arr[:20])

# Copy to GPU
d_input = cuda.to_device(arr)

threads_per_block = 256

# Multi-pass reduction
while True:
    blocks = math.ceil(n / threads_per_block)

    d_output = cuda.device_array(blocks, dtype=np.int32)

    # Launch kernel
    reduce_kernel[blocks, threads_per_block, 0, threads_per_block * 4]
(d_input, d_output, n)

    if blocks == 1:
        break

    # Prepare for next iteration
    d_input = d_output
    n = blocks

# Final result
result = d_output.copy_to_host()

print("Final Sum:", result[0])
```

```
Enter number of elements: 10345
Generated array (first 20 elements): [ 5  7  1  8  4 10  2  6  9  2  2
3  4 10  4 10  7  8  3  1]
Final Sum: 56825

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 41 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/dispatch
er.py:696: NumbaPerformanceWarning: Grid size 1 will likely result in
GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
```

```python
# Bitonic Sort using CUDA in Python (Numba)

import numpy as np
from numba import cuda
import math
import time

# CUDA Kernel for Bitonic Sort
@cuda.jit
def bitonic_sort_step(dev_values, j, k):
    i = cuda.grid(1)

    ixj = i ^ j

    if ixj > i:
        # Ascending order
        if (i & k) == 0:
            if dev_values[i] > dev_values[ixj]:
                temp = dev_values[i]
                dev_values[i] = dev_values[ixj]
                dev_values[ixj] = temp

        # Descending order
        if (i & k) != 0:
            if dev_values[i] < dev_values[ixj]:
                temp = dev_values[i]
                dev_values[i] = dev_values[ixj]
                dev_values[ixj] = temp


# Main Function
def bitonic_sort(arr):

    n = len(arr)

    # Copy data to GPU
    dev_arr = cuda.to_device(arr)

    threads_per_block = 256
    blocks = (n + threads_per_block - 1) // threads_per_block

    # Bitonic Sort
    k = 2
    while k <= n:

        j = k // 2

        while j > 0:
            bitonic_sort_step[blocks, threads_per_block](dev_arr, j,
k)
```

```
            cuda.synchronize()

            j = j // 2

        k = k * 2

    # Copy sorted data back to CPU
    sorted_arr = dev_arr.copy_to_host()

    return sorted_arr


# Driver Code
if __name__ == "__main__":

    # Size must be power of 2
    N = 16

    arr = np.random.randint(0, 100, N).astype(np.int32)

    print("Original Array:")
    print(arr)

    start = time.time()

    sorted_arr = bitonic_sort(arr)

    end = time.time()

    print("\nSorted Array:")
    print(sorted_arr)

    print("\nExecution Time:", end - start, "seconds")
```

```
Original Array:
[70 63 56 71 31 89 38 43 92 73 44 98 72 96 43 27]

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))


Sorted Array:
[27 31 38 43 43 44 56 63 70 71 72 73 89 92 96 98]

Execution Time: 2.3781802654266357 seconds
```

```python
# Histogram Calculation using CUDA in Python (Numba)

import numpy as np
from numba import cuda
import math
import time

# CUDA Kernel for Histogram
@cuda.jit
def histogram_kernel(data, histo):

    idx = cuda.grid(1)

    # Check bounds
    if idx < data.size:

        value = data[idx]

        # Atomic add to avoid race condition
        cuda.atomic.add(histo, value, 1)


# Main Function
def compute_histogram(data, bins):

    # Allocate histogram array
    histo = np.zeros(bins, dtype=np.int32)

    # Copy arrays to GPU
    d_data = cuda.to_device(data)
    d_histo = cuda.to_device(histo)

    # CUDA Configuration
    threads_per_block = 256
    blocks_per_grid = math.ceil(data.size / threads_per_block)

    # Launch Kernel
    histogram_kernel[blocks_per_grid, threads_per_block](d_data,
d_histo)

    # Copy result back to CPU
    result = d_histo.copy_to_host()

    return result


# Driver Code
if __name__ == "__main__":

    # Random data between 0 and 9
    data = np.random.randint(0, 10, size=1000).astype(np.int32)
```

```
    bins = 10

    print("Input Data:")
    print(data)

    start = time.time()

    histogram = compute_histogram(data, bins)

    end = time.time()

    print("\nHistogram Result:")
    for i in range(bins):
        print(f"Bin {i}: {histogram[i]}")

    print("\nExecution Time:", end - start, "seconds")
```

```
Input Data:
[3 6 2 0 0 5 7 6 1 9 1 7 5 2 7 1 9 0 9 5 6 6 1 6 3 2 5 8 6 1 6 3 5 6 6
7 6
 1 8 2 7 0 9 2 0 4 4 0 7 0 9 5 9 8 5 4 6 0 7 2 9 6 4 9 1 3 2 4 0 9 3 0
2 1
 9 8 5 2 1 7 6 0 8 9 6 2 8 0 1 3 4 2 0 7 7 2 8 6 8 8 3 4 1 1 4 2 3 9 9
3 4
 6 3 3 7 4 9 1 4 9 3 2 1 2 7 0 2 8 9 2 2 6 0 2 4 8 9 4 7 5 2 2 5 3 2 9
5 2
 2 5 4 0 0 7 2 7 1 1 6 4 8 6 3 9 2 7 8 7 1 0 5 9 2 4 8 3 1 6 5 0 1 2 0
4 9
 6 1 5 0 4 0 9 9 9 5 2 9 3 1 4 3 4 1 0 8 0 9 5 9 2 9 3 2 0 2 4 0 7 2 6
3 2
 6 8 5 1 3 7 0 3 2 6 3 8 1 6 0 9 1 8 4 9 0 0 7 9 5 9 2 2 6 2 5 9 6 9 3
4 6
 5 8 1 8 5 5 3 0 8 2 9 3 5 8 4 6 9 0 3 5 9 0 6 4 8 3 2 6 8 2 9 7 6 5 3
9 1
 7 3 9 4 1 7 7 6 4 1 6 9 3 4 7 6 6 3 4 4 3 7 3 9 0 4 5 2 5 4 9 5 6 1 2
9 5
 0 2 1 6 6 6 4 0 3 8 2 3 7 3 7 3 8 8 3 5 7 9 4 4 0 3 7 6 0 3 5 5 1 3 6
1 5
 3 3 8 2 8 0 7 6 1 2 9 6 5 9 9 5 5 5 0 2 4 2 0 1 1 5 2 3 1 2 6 4 0 6 3
4 2
 4 5 8 4 3 4 8 0 2 6 3 7 6 4 0 2 7 2 9 4 6 4 1 4 4 1 2 3 9 3 8 6 6 5 6
1 3
 8 5 3 9 4 2 4 8 5 1 5 3 5 7 3 5 4 9 0 6 2 8 1 7 3 9 3 3 3 6 4 9 5 9 3
8 3
 4 5 6 7 8 9 9 4 2 8 0 1 0 6 3 8 6 4 5 5 6 8 3 4 3 7 5 9 4 5 0 0 8 6 5
7 8
 9 5 1 9 9 6 5 4 9 9 5 4 8 3 8 4 1 8 1 5 5 2 5 7 3 0 1 0 5 7 3 0 4 6 8
7 7
 1 4 5 4 5 7 7 9 3 9 5 7 9 8 4 2 3 6 6 6 8 3 5 0 8 0 9 6 5 7 5 0 9 7 6
```

```
2 1
 6 2 3 9 1 8 7 7 5 0 3 5 5 7 2 6 4 5 1 1 9 8 7 9 8 5 2 4 5 1 7 2 8 4 0
3 8
 1 5 0 1 4 2 5 5 3 1 8 9 9 3 8 3 2 8 1 7 0 1 0 4 7 8 1 8 0 8 5 2 8 5 4
5 1
 8 4 4 5 3 5 1 7 9 5 3 8 8 2 5 1 6 3 6 3 8 7 9 3 0 2 7 9 4 9 1 7 0 6 7
9 7
 3 8 7 7 4 2 2 6 7 9 2 6 9 5 1 5 4 6 0 6 6 6 0 1 2 7 0 9 1 6 8 9 7 5 8
7 4
 9 8 7 0 4 1 0 9 6 5 8 9 6 8 1 9 8 0 0 2 0 1 1 1 1 2 4 5 4 1 9 2 1 3 3
5 1
 5 1 7 5 3 0 3 0 1 4 6 9 6 1 5 5 7 5 1 3 8 5 8 5 2 5 9 8 4 7 5 8 5 4 4
2 7
 1 6 5 6 9 4 0 7 9 5 0 4 4 8 6 4 0 4 6 7 2 9 4 4 0 3 1 2 7 5 4 3 4 3 4
5 1
 2 5 0 8 6 1 5 8 4 9 0 8 3 2 8 3 2 6 0 9 6 0 7 9 0 9 7 2 8 0 4 7 5 9 6
3 8
 2 9 6 9 6 2 7 1 7 4 2 3 5 6 4 3 7 2 5 7 2 9 3 1 9 0 2 1 3 2 6 3 0 6 6
1 8
 7 1 6 3 4 5 7 3 5 7 5 9 1 1 9 4 0 8 3 2 7 6 9 7 3 6 7 2 4 9 1 1 7 6 7
4 9
 7 1 0 3 8 5 8 7 4 2 2 5 2 9 9 8 5 5 3 8 4 9 6 3 6 9 5 4 3 5 9 7 9 7 4
9 7
 9]

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 4 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))


Histogram Result:
Bin 0: 87
Bin 1: 93
Bin 2: 98
Bin 3: 104
Bin 4: 103
Bin 5: 116
Bin 6: 102
Bin 7: 94
Bin 8: 89
Bin 9: 114

Execution Time: 2.6657469272613525 seconds
```

```python
# CUDA Memory Management using Local and Shared Memory in Python
(Numba)

import numpy as np
from numba import cuda, float32
import math

# CUDA Kernel
@cuda.jit
def memory_demo(input_array, output_array):

    # Shared Memory (shared by threads in a block)
    shared_mem = cuda.shared.array(shape=256, dtype=float32)

    # Thread index
    tid = cuda.threadIdx.x

    # Global index
    idx = cuda.grid(1)

    # ---------------- LOCAL MEMORY ----------------
    # Local variable (private to each thread)
    local_var = 0.0

    if idx < input_array.size:

        # Store data into local memory
        local_var = input_array[idx] * 2

        # Store into shared memory
        shared_mem[tid] = local_var

    # Synchronize threads in block
    cuda.syncthreads()

    # Read from shared memory
    if idx < input_array.size:

        output_array[idx] = shared_mem[tid] + 5


# Driver Code
if __name__ == "__main__":

    N = 256

    # Input Data
    input_data = np.arange(N).astype(np.float32)

    # Output Array
    output_data = np.zeros_like(input_data)
```

```
    # Copy data to GPU
    d_input = cuda.to_device(input_data)
    d_output = cuda.to_device(output_data)

    # CUDA Configuration
    threads_per_block = 256
    blocks_per_grid = math.ceil(N / threads_per_block)

    # Launch Kernel
    memory_demo[blocks_per_grid, threads_per_block](d_input, d_output)

    # Copy result back to CPU
    result = d_output.copy_to_host()

    print("Input Array:")
    print(input_data[:10])

    print("\nOutput Array:")
    print(result[:10])
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))

Input Array:
[0. 1. 2. 3. 4. 5. 6. 7. 8. 9.]

Output Array:
[ 5.  7.  9. 11. 13. 15. 17. 19. 21. 23.]
```

```python
# Parallel Reduction Multiplication using CUDA in Python (Numba)

import numpy as np
from numba import cuda, float32
import math
import time

# CUDA Kernel for Parallel Reduction Multiplication
@cuda.jit
def reduction_multiply(input_array, output_array):

    # Shared memory
    shared_data = cuda.shared.array(256, dtype=float32)

    tid = cuda.threadIdx.x
    idx = cuda.grid(1)

    # Load data into shared memory
    if idx < input_array.size:
        shared_data[tid] = input_array[idx]
    else:
        shared_data[tid] = 1.0

    # Synchronize threads
    cuda.syncthreads()

    # Parallel Reduction Multiplication
    stride = cuda.blockDim.x // 2

    while stride > 0:

        if tid < stride:
            shared_data[tid] *= shared_data[tid + stride]

        cuda.syncthreads()

        stride //= 2

    # Store block result
    if tid == 0:
        output_array[cuda.blockIdx.x] = shared_data[0]


# Driver Code
if __name__ == "__main__":

    N = 256

    # Input Array
    input_data = np.arange(1, N + 1).astype(np.float32)
```

```
    print("Input Array:")
    print(input_data[:10])

    # Output array for block results
    output_size = math.ceil(N / 256)
    output_data = np.zeros(output_size, dtype=np.float32)

    # Copy to GPU
    d_input = cuda.to_device(input_data)
    d_output = cuda.to_device(output_data)

    threads_per_block = 256
    blocks_per_grid = output_size

    start = time.time()

    # Launch Kernel
    reduction_multiply[blocks_per_grid, threads_per_block](d_input,
d_output)

    cuda.synchronize()

    end = time.time()

    # Copy result back
    partial_result = d_output.copy_to_host()

    # Final multiplication on CPU
    final_result = np.prod(partial_result)

    print("\nPartial Block Products:")
    print(partial_result)

    print("\nFinal Product:")
    print(final_result)

    print("\nExecution Time:", end - start, "seconds")
```

```
Input Array:
[ 1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]

/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))


Partial Block Products:
[inf]

Final Product:
```

```
inf

Execution Time: 2.142974853515625 seconds
```

```
# Generate Square Star Pattern in Parallel using CUDA (Numba)

from numba import cuda
import numpy as np

# CUDA Kernel
@cuda.jit
def square_pattern(pattern):

    row, col = cuda.grid(2)

    n = pattern.shape[0]

    # Boundary check
    if row < n and col < n:

        # Fill with '*'
        pattern[row][col] = 1


# Driver Code
if __name__ == "__main__":

    N = 8   # Size of square

    # Create empty matrix
    pattern = np.zeros((N, N), dtype=np.int32)

    # Copy to GPU
    d_pattern = cuda.to_device(pattern)

    # Threads and Blocks
    threads_per_block = (16, 16)

    blocks_per_grid_x = (N + threads_per_block[0] - 1) //
threads_per_block[0]
    blocks_per_grid_y = (N + threads_per_block[1] - 1) //
threads_per_block[1]

    blocks_per_grid = (blocks_per_grid_x, blocks_per_grid_y)

    # Launch Kernel
    square_pattern[blocks_per_grid, threads_per_block](d_pattern)

    # Copy back to CPU
    result = d_pattern.copy_to_host()

    # Print Pattern
    print("Square Star Pattern:\n")

    for i in range(N):
```

```
        for j in range(N):

            if result[i][j] == 1:
                print("*", end=" ")
            else:
                print(" ", end=" ")

        print()
```

```
/usr/local/lib/python3.12/dist-packages/numba_cuda/numba/cuda/
dispatcher.py:696: NumbaPerformanceWarning: Grid size 1 will likely
result in GPU under-utilization due to low occupancy.
  warn(errors.NumbaPerformanceWarning(msg))
```

```
Square Star Pattern:

* * * * * * * *
* * * * * * * *
* * * * * * * *
* * * * * * * *
* * * * * * * *
* * * * * * * *
* * * * * * * *
* * * * * * * *
```